

This is a powerful extension of your current platform. Using Gemini Live API for this is a smart move because its low latency and native tool-calling capabilities are perfect for real-time UI manipulation.

To ensure we build this SDK correctly, I need to clarify the "handshake" between your AI and the developer's code. Here are 20 questions:

The Developer Experience (Integration)

1. NPM vs. Script Tag: Will the SDK be primarily an NPM package for React/Next.js users, or a CDN `<script>` tag for plain HTML projects?
2. Initialization: How does the SDK identify which agent to load? (e.g., `Doweit.init({ apiKey: '...', agentId: '...' })`)?
3. Action Mapping: How does the developer "link" an AI intent to a JavaScript function? (e.g., do they pass an object of functions like `{ navigateTo: (path) => router.push(path) }`?)
4. Intent Definition: Will developers define intents/commands on your Dashboard (visual UI) or by writing a JSON/Script file in their own code?
5. Dynamic Context: Can the developer send the "current app state" to the AI? (e.g., "The user currently has 3 items in their cart").

The AI Logic (Gemini Live API)

6. Function Calling: Will we use Gemini's native `tool_use` (Function Calling) to trigger the SDK actions?
7. Latency Management: Gemini Live requires a WebSocket. Will the client browser connect directly to Google (using a client-side key) or proxy through your backend to hide your API keys?
8. Context Window: How will the AI know what pages exist on the developer's site? Will the developer provide a "Sitemap" in the dashboard?
9. Language Support: Will the SDK detect the browser language automatically or follow the setting in the Doweit Dashboard?

The Widget UI (The "Bubble")

10. Customization: Beyond colors and position, can the developer provide a custom React component for the chat bubble, or is it strictly your pre-designed UI?
11. Text vs. Voice: How will the UI transition between "Voice Mode" and "Text

Mode"? Will there be a chat history window that slides up?

12. Visual Feedback: When the AI is "Thinking" or "Executing an action," how will the widget visualize that to the end-user?

Interactions & Constraints

13. Wake Word: Will it support a wake word like "Hey Doweit" or only trigger on a button click?

14. Navigation: If the AI says "Taking you to the pricing page," does the SDK handle the URL change, or does it just send a "signal" that the developer must listen for?

15. Data Extraction: If a user says "Book a table for 5 people," will the SDK return structured JSON ({ "guests": 5 }) to the developer's app?

16. Security: How do we prevent a user from "Prompt Injecting" the SDK to trigger unauthorized functions (e.g., "Delete my account")?

The Dashboard (Doweit Admin)

17. Testing Sandbox: Will your dashboard have a "Simulator" where the developer can type/speak to see which function would have been triggered?

18. Usage Analytics: Will you track how many "actions" were triggered per developer for billing purposes?

19. Avatar Customization: Will the widget support 3D avatars (like Alan AI) or simple 2D images/animations?

20. Version Control: If a developer changes their "Dialog Script" in your dashboard, does it update the live website immediately?

Once you answer these, I can start mapping out the doweit-sdk.js file and the Dashboard logic.

1. lets make both for simpler users they can use the script but we will primarily focust on the npm package road
2. yeah based on what the developer choose the agent i mean in the doweit alan ai it will give them that which they will take and put there
3. The Actual Flow
4. Define the intent in Alan script

In Alan, you write something like:

```
intent('Open dashboard', p => { p.play({ command: 'openDashboard' }); });
```

👉 This says: “When user says Open dashboard, send a command called openDashboard to the app.”

2. Catch it in your JavaScript app

In your frontend (React / Next.js / plain JS), you listen for that command:

```
alanBtn({ key: 'YOUR_KEY', onCommand: (commandData) => { if
(commandData.command === 'openDashboard') { openDashboard(); // your JS
function } } });
```

👉 This is where the “link” happens:

command: 'openDashboard' (Alan side) if (command === 'openDashboard') (JS side)

the above is how alan ai does it usually but am thinking of making it better this way

🚫 Where Alan AI starts to break down

If you’ve used it enough, you’ve probably felt this:

You must predefine intents (“Open dashboard”, “Go to dashboard”, etc.) Logic is split between Alan script + frontend Hard to handle complex, multi-step requests Context is shallow (doesn’t “think” across actions well) Feels more like voice commands than actual AI 🚀 What a better version would look like

1. 🧠 Intent-less architecture (this is the big one)

Instead of:

```
intent('Open dashboard', ...)
```

You move to:

“Here are all the things my app can do”

Example:

```
actions = [ { name: "openDashboard", description: "Navigates user to dashboard" },  
  { name: "playVideo", description: "Plays the current video" } ]
```

Then the AI decides:

what the user means which action to call with what parameters

👉 This is closer to how modern AI agents work (function calling)

2. 🔗 Direct function binding (no string matching)

Instead of this fragile link:

```
if (command === 'playVideo')
```

You expose functions directly:

```
ai.register({ playVideo: () => video.play(), pauseVideo: () => video.pause() });
```

The AI calls:

```
{ "action": "playVideo" }
```

👉 No string duplication, no mismatch bugs

3. 🧠 Persistent memory + context engine

Alan is mostly stateless per command.

A better system would:

remember what the user is doing track UI state understand flows

Example:

“Play something” → picks a movie “Pause it” → knows what “it” is “Resume in 10 seconds” → schedules action

4. 🎯 Multi-step reasoning (agent behavior)

Instead of 1 command → 1 action:

for example in a restaurant app i told you before in that example

User:

“Find me the most expensive burger and add 2 in my cart ”

AI should:

search content choose a result select 2 add to cart

👉 That’s not an “intent”, that’s a plan

4. If you force developers to pick only one, you’ll end up limiting your product.

The better move is: support both—but make one the “power path” and the other the “easy path.”

🎯 The real tradeoff 📄 Option 1: JSON / Script (in their codebase)

Pros

Version controlled (Git) Fits dev workflows Easy to review, test, CI/CD Scales well for complex apps

Cons

Slower onboarding Feels “heavy” for simple use cases 💻 Option 2: Visual Dashboard (no-code / low-code)

Pros

Fast onboarding (“it just works”) Great for demos and non-devs Easy to tweak behavior live

Cons

Becomes messy at scale Hard to track changes Devs don’t fully trust it long-term 🚀

What a better-than-Alan product should do 👉 1. Make code the source of truth

Let developers define capabilities in code:

```
ai.register({ playVideo: { description: "Play the current video", handler: () =>
video.play() } });
```

This should be the real, authoritative layer.

👉 2. Generate the Dashboard automatically

Your UI shouldn't replace code — it should reflect it.

So the dashboard:

Reads registered actions Shows them visually Lets you tweak: descriptions examples permissions testing

👉 Think: DevTools for your AI, not a replacement for code

👉 3. Allow optional visual overrides (but sync back)

If someone edits in the UI:

it updates the underlying config (JSON or schema) can be exported back to code

No “hidden logic” trapped in your dashboard.

👉 4. Add a playground (this is HUGE)

Where your product wins:

A live panel where devs can:

type or speak see: user input interpreted intent chosen action parameters execution result

👉 This solves the biggest pain Alan has: debugging

💡 The winning combo (this is the answer)

Primary: Code-first (JSON / SDK) Secondary: Visual dashboard (auto-generated + editable)

5. Short answer: yes—and it shouldn't just be possible, it should be a core feature. But the way you let developers do it is what will make your product feel next-level.

💡 The problem you're solving

Without context, AI is guessing.

With context:

“Checkout my cart” becomes “User has 3 items → total €120 → suggest checkout or upsell”

So the AI stops being a voice trigger and becomes aware of the app state.

🚫 What not to do

Don't make devs manually inject context into prompts like:

```
ai.setContext("User has 3 items in cart...");
```

That turns into:

messy strings repeated logic hard-to-maintain code 🚀 What's better (this is your opportunity) 👉 1. Structured state, not text

Let devs send real objects, not sentences:

```
ai.setState({ cart: { items: 3, total: 120 }, user: { isPremium: true } });
```

👉 The AI understands structured data far better than raw text.

👉 2. Make it reactive (this is 🔥)

Instead of manually updating:

```
ai.setState(...)
```

Let devs bind state:

```
ai.bindState(() => ({ cart: getCart(), currentVideo: videoState, roomUsers: users }));
```

Now:

whenever state changes → AI stays updated automatically no extra calls needed 👉 3. Scoped context (super important)

Not all context should always be visible.

Let devs define scopes:

```
ai.setState({ global: {...}, session: {...}, screen: {...} });
```

Example:

global → user profile screen → current page data session → temporary stuff

👉 Prevents overload + improves accuracy

👉 4. Let AI query state instead of dumping everything

Even better approach:

```
ai.registerState({ getCart: () => cart, getUsers: () => users });
```

Then AI can decide:

when it needs data what to fetch

👉 This avoids sending unnecessary data constantly

👉 5. Combine with actions (this is where magic happens)

Now AI can:

read state decide act

Example flow if the developer has for example a virtual cinema movie site:

user : - “Start the movie if everyone is ready”

AI:

checks roomUsers verifies readiness calls startVideo()

👉 That’s agent behavior, not commands

The AI LOGIC

🧠 1. Function Calling (Gemini tool_use)

Yes — you should absolutely use Gemini’s native function calling.

That’s your foundation.

Why this is the right move Removes fragile intent matching completely Lets the model decide the action + parameters Scales naturally to complex flows What it should look like in your system

Developer defines:

```
ai.register({ playVideo: { description: "Play the current video", params: {} } });
```

You send to Gemini:

```
{ "tools": [ { "name": "playVideo", "description": "Play the current video" } ] }
```

Gemini responds:

```
{ "tool_use": { "name": "playVideo", "args": {} } }
```

👉 Your SDK executes it.

🔥 Key upgrade (this is where you beat others)

Don't just execute immediately.

Add a middleware layer:

```
validate log allow/deny modify args ai.use((action, next) => { if (action.name ===  
"deleteAccount") return; next(); });
```

Now you have control + safety.

⚡ 2. Latency & WebSocket Architecture

This one is critical. If you get it wrong, your product is dead on arrival.

❌ Do NOT connect browser → Gemini directly

Even if it's tempting.

Why: API key exposure (huge risk) No control over usage / billing No logging or debugging Can't inject system prompts properly Can't enforce permissions ☒ Correct architecture (production-grade)

Browser → Your Backend → Gemini Live (WebSocket)

Client (SDK) ↓ Your Backend (WebSocket proxy) ↓ Gemini Live API What your backend should handle: API key protection session management injecting: tools context system instructions streaming responses back to client logging + analytics 🔥 Pro move (big differentiator)

Create a session layer:

```
const session = await ai.createSession({ userId, appId });
```

Now you can:

persist memory track conversations replay/debug sessions 🧠 3. Context Window /
“How does AI know the app?”

This is where most products fail.

❌ Don't rely on a sitemap alone

A sitemap tells you:

pages exist

But NOT:

what users can do what actions mean what UI state is ✅ The correct model:
“Capability Graph”

You want 3 layers:

🧩 Layer 1: Actions (most important)

Defined in code:

```
ai.register({ openDashboard: {...}, playVideo: {...}, sendMessage: {...} });
```

👉 This is what the AI actually uses

📖 Layer 2: App Structure (optional but useful)

Yes, include something like a sitemap:

```
{ "pages": [ { "name": "Dashboard", "route": "/dashboard" }, { "name": "Video Room",  
"route": "/room/:id" } ] }
```

👉 Helps with navigation language:

“go to dashboard” “open room 3” ⚡ Layer 3: Live Context (the real power)

From earlier:

```
ai.setState({ currentPage: "videoRoom", videoPlaying: true, usersInRoom: 5 }); 🧠 How  
it all comes together
```

When sending to Gemini, you combine:

tools (actions) system prompt (rules) current state minimal app structure

👉 Not everything. Only what's relevant.

🔥 Advanced idea (this is big)

Instead of sending full sitemap, generate:

“Current UI context”

Example:

```
{ "screen": "Video Room", "availableActions": ["playVideo", "pauseVideo",  
"sendMessage"] }
```

👉 This dramatically improves accuracy and reduces token usage.

🧠 Final Architecture Summary ✅ Function Calling Use Gemini tool_use Add
middleware layer (critical) ✅ Networking NEVER expose API key Always proxy through
backend Add session system ✅ Context Not just sitemap Use: actions (core) state
(dynamic) UI context (smart filtering) 💡 If you want to build something truly better

Your edge is not:

“using Gemini”

Your edge is:

how cleanly developers can expose their app to the AI

4. it will follow the setting in the dashboard when the user customizes the agent he will define this for the agent like choose up to 3 languages per agent the agent won't answer for another language other than those 3

The Widget UI

1. for now let's only go with the pre-designed UI (because latter I will keep on working on more UI widget types for the chat bubble the user can choose from)
2. the developer will set this up in the dashboard toggle the text and audio mode if the developer chooses both. Single conversation UI (chat-first)

Have one unified chat interface:

Messages (user + assistant) Works for both voice and text Persistent history

👉 Yes, include a chat history panel — this is essential.

3. using a beautiful loader that can be chosen on the dashboard again from the list of loader options

Interactions & Constraints

1. on the dashboard we will put the wake word functionality as coming soon for now let's just use the button click one to make it easier
2. 🌀 1. Navigation

Should the SDK change the URL, or just send a signal?

✅ Best answer: SDK sends an intent signal — developer owns navigation Why this is the right design Every app routes differently (Next.js, React Router, native, etc.) You avoid breaking app state or custom guards Devs keep full control (auth, analytics, transitions) 🔧 What it looks like

AI decides:

```
{ "tool_use": { "name": "navigate", "args": { "route": "/pricing" } } }
```

SDK emits:

```
ai.onAction((action) => { if (action.name === "navigate")  
{ router.push(action.args.route); } }); 🔥 Upgrade (makes your product feel premium)
```

Provide optional helpers, not hard control:

```
ai.enableNavigation(router);
```

👉 Internally maps:

```
navigate → router.push()
```

So:

beginners → plug & play advanced devs → full control 🧩 2. Data Extraction

“Book a table for 5 people” → { guests: 5 }

✅ Best answer: Yes — always return structured JSON

This is where you move beyond Alan-style systems.

🔧 Developer defines schema: `ai.register({ bookTable: { description: "Reserve a table at the restaurant", params: { guests: "number", date: "string", time: "string" } } });` 🧠 AI returns: `{ "tool_use": { "name": "bookTable", "args": { "guests": 5, "time": "7pm" } } }` 💡 SDK guarantees: Type validation Missing field handling Default values (optional) 🔥 Big differentiator

Add auto-clarification:

If user says:

“Book a table”

AI should respond:

“For how many people?”

👉 Don't call the function until required params exist

🔒 3. Security (Prompt Injection Protection)

This is the most important one—and most people get it wrong.

🚫 The wrong approach

“Just trust the AI to behave.”

Nope.

✅ The correct approach: multi-layer defense 🧱 Layer 1: Explicit Action Permissions

Every action must declare:

`ai.register({ deleteAccount: { dangerous: true, requireConfirmation: true } });` 🔴 Layer 2: Middleware (your control layer) `ai.use((action, next) => { if (action.name === "deleteAccount") { if (!user.isVerified) return; } next(); });`

👉 Nothing executes without passing here

🔒 Layer 3: Server-side enforcement (critical)

Even if frontend is tricked:

backend must re-check permissions

👉 Never trust client-triggered actions

⚠️ Layer 4: AI instruction hardening

Your system prompt should clearly define:

AI cannot invent actions AI can only use registered tools AI must follow permission rules 🧐 Layer 5: Action allowlist (simple but powerful)

Only allow execution of:

explicitly registered tools

👉 If AI tries:

```
{ "name": "hackSystem" }
```

→ ignored instantly

🔥 Layer 6: Confirmation UX (for sensitive actions)

User says:

“Delete my account”

System responds:

“Are you sure you want to delete your account?”

Only proceeds after:

explicit confirmation

The Dashboard

1. yes exactly and this has to be the most beautiful and perfect interaction and very functional
2. yeah that would be perfect
3. no 3d avatars for now simple 2d animations
4. Short answer: don't make dashboard edits go live instantly by default. That feels convenient early on, but it will bite developers later.

The better approach is a versioned + staged deployment model—just like code.

🧠 The core problem

If changes go live immediately:

😬 You can break production accidentally 😬 No rollback 😬 No testing 😬 Teams step on each other

That's fine for demos, not for real apps.

✅ What you should build instead

1. Draft → Publish flow (baseline)

Every change in your dashboard creates a draft version.

Nothing goes live until the developer clicks Publish.

Example flow Dev edits action / prompt / behavior Saved as Draft v12 Tests it in playground Clicks Publish → becomes Live v12 2. Environments (this is big)

Support:

Development Staging Production

👉 Each environment has its own version

Example: Dev tweaks behavior → goes to Development QA tests → promote to Staging Ready → promote to Production 3. Version history + rollback

Every publish creates a version:

v10 → v11 → v12

If something breaks: 👉 one click → rollback to v11

4. Optional “auto-deploy” (for small teams)

For beginners, you can allow:

“Auto-publish changes”

But:

make it opt-in clearly labeled as risky 5. Sync with code (important for trust)

If your system has both:

dashboard edits code-based config

You must avoid conflicts.

Two good options: 🦋 Option A (recommended): Code = source of truth Dashboard edits generate config Dev exports to code (JSON/SDK) Deploy via Git

👉 Most scalable, dev-friendly

🦋 Option B: Cloud-managed config Dashboard is source of truth SDK pulls latest published version

👉 Faster, but less control

🔥 Best hybrid (this is ideal) Dashboard = editor + testing Code = deployment authority

Flow:

Edit in dashboard Test in playground Export config Commit to repo Deploy normally ⚡
Bonus: "Preview Mode" (huge UX win)

Let developers test drafts live without publishing:

?ai_preview=true

👉 Only they see the new behavior 👉 Users still see stable version

💡 Final recommendation Default behavior: ❌ No instant live updates ✅ Draft →
Publish required Advanced: ✅ Environments ✅ Version history ✅ Rollback ✅
Preview mode

This is a brilliant roadmap. You are essentially building a "Headless AI OS" for websites where the code is the source of truth, and the dashboard is the management and debugging suite. By moving to an Intent-less (Capability-based) Architecture, you are making the developer's life 10x easier.

To lock in the final "handshake" between the SDK, your Backend, and Gemini Live, here are my final 10 questions:

The Technical Handshake

1. Action Return Values: When Gemini calls a tool (e.g., `bookTable`), the SDK executes the JS function. How does the AI receive the result of that function (e.g., `{ "status": "success", "reservationId": "123" }`) so it can tell the user "I've booked your table, your ID is 123"?
2. State Sync Strategy: If a developer uses `ai.bindState` on a fast-changing value (like a countdown timer or a search input), should the SDK send every update to your backend immediately, or should the AI "request" a state snapshot only when it needs to make a decision?
3. Interruption Handling: Gemini Live handles voice interruptions natively, but on the frontend, if the AI is currently playing a long audio response and the user starts speaking, should the SDK immediately kill the local audio context and "listen"?
4. End-User Identity: How will your backend track User Memory across sessions? Will the SDK pass an `anonymousId` (stored in cookies/localStorage) or an `authenticatedUserId` from the developer's system?

Development & Deployment

5. Dynamic Action Registration: Can a developer register actions on-the-fly? (e.g., only register the checkout action when the user actually navigates to the `/cart` page)?

6. Parameter Clarification: If an action requires 3 parameters but the user only provides 1, do you want the Gemini System Prompt to handle the "wait, I need more info" logic, or should the SDK have a "Required Fields" validation layer that prevents the function from being called?
7. The "Code-Truth" Sync: When a developer registers an action in code, how does the Doweit Dashboard learn about it to show it in the UI? Will the SDK send a "Manifest" of all registered functions to your backend during initialization?

Edge Cases & UX

8. The "I Don't Know" Fallback: If a user asks for something the app cannot do (no action matches), should the AI simply chat like a generic assistant, or should it be strictly limited to only talking about the app's capabilities?
9. Confirmations UI: For "dangerous" actions (like deleteAccount), will you provide a pre-built UI prompt (a popup inside the widget) that the user must click, or must the developer build that confirmation modal themselves?
10. Multi-Language Switching: Since you support up to 3 languages, will the AI try to auto-detect which one the user is speaking mid-conversation, or must the user/developer explicitly set the active language?

Once these are answered, we can start the implementation. I suggest we start with the "Manifest" logic (how the SDK tells the backend what it can do) and the WebSocket Proxy.

The Technical Handshake

1. 1. Action Return Values

How does Gemini receive the result of `bookTable()`?

- ✅ Best approach: Tool result round-trip via backend (not frontend-only)

You need a full loop:

Step 1 — AI calls tool

```
{  
  
  "tool_use": {  
  
    "name": "bookTable",  
  
    "args": { "guests": 5 }  
  
  }  
  
}
```

Step 2 — SDK executes function

```
const result = await bookTable({ guests: 5 });
```

Example result:

```
{  
  
  "status": "success",  
  
  "reservationId": "123"  
  
}
```

Step 3 — SDK sends result BACK to Gemini

This is the critical missing piece in most systems.

```
{  
  "tool_result": {  
    "name": "bookTable",  
    "result": {  
      "status": "success",  
      "reservationId": "123"  
    }  
  }  
}
```

Step 4 — Gemini continues reasoning

Now Gemini can say:

“Done — your table is booked, reservation ID is 123.”

🔥 Key insight

The AI is not calling your JS directly.

It's a loop:

AI → tool call → SDK → result → AI → response

⚡ 2. State Sync Strategy

Push everything or request-on-demand?

❌ Do NOT push every update to backend

That will:

explode bandwidth

kill latency

waste tokens

create noise in context

✅ Best architecture: Hybrid (event + pull model)

🧠 Rule:

State is stored locally. AI only receives it when needed.

Option A — Reactive binding (light sync)

```
ai.bindState(() => ({  
  cart: getCart(),  
  page: currentPage  
}));
```

👉 SDK caches this locally

Option B — On-demand fetch (preferred for fast-changing data)

Instead of pushing constantly:

```
ai.registerState({  
  getCart: () => cart,  
  getSearchQuery: () => inputValue  
});
```

 What Gemini actually sees

Only when needed:

```
{  
  "cart": { "items": 3 },  
  "page": "checkout"  
}
```

 Best practice (important)

Split state into 3 categories:

Type	Strategy
------	----------

Static (user profile)	cached
-----------------------	--------

Session (cart, page)	snapshot on demand
----------------------	--------------------

Ephemeral (typing, timer)	only query when needed
---------------------------	------------------------

🚫 Never do

sending every keystroke to backend

streaming UI state continuously

🎤 3. Interruption Handling (barge-in UX)

✅ Yes — SDK should immediately interrupt audio

But do it cleanly.

🧠 Required behavior:

When user starts speaking:

stop audio playback instantly

cancel TTS stream

switch to listening mode

optionally notify backend

Example flow:

```
audio.stop();
```

```
voice.startListening();
```

🔥 Important detail: “soft interrupt vs hard interrupt”

Hard interrupt (default)

immediately kills audio

switches to mic input

Soft interrupt (advanced)

fades audio

pauses response

resumes if user stops talking

💡 Pro UX upgrade

Add:

“AI is speaking... tap to interrupt”

But also allow:

voice interruption without touch (natural barge-in)

🧠 Key principle

The fastest input always wins (voice > audio > text)

👤 4. End-User Identity (memory system)

This is where most SDKs get this wrong.

You need two identity modes.

🗂️ Option A — Anonymous ID (default)

SDK generates:

```
anonymousId = "a8f92k3..."
```

Stored in:

localStorage

cookie fallback

Pros:

no login required

instant onboarding

works for 90% apps

 Option B — Authenticated ID (recommended for real apps)

Developer passes:

```
ai.setUser({  
  userId: "user_123",  
  email: "john@site.com"  
});
```

 Backend behavior:

You map:

user_123 → memory, history, preferences

🔥 Best architecture: dual identity linking

If user logs in later:

anonymousId → merged into userId

So memory is preserved.

🚨 Critical security rule

Never trust:

client-provided identity for authorization

only for personalization

Authorization must still be server-side.

Development & Deployment

1. 🧩 1. Dynamic Action Registration

Can actions be registered on-the-fly (e.g. only on /cart)?

✅ Yes — but with a strict model: scoped action registry

You should support dynamic registration, but not as random runtime mutations.
Instead, think in scopes + lifecycle.

💡 Best design: “Scoped Actions”

```
ai.register({  
  
  checkout: {  
  
    scope: "/cart",  
  
    description: "Process user checkout"  
  
  }  
  
});
```

🔄 Runtime behavior

When user enters /cart:

actions with scope: "/cart" become active

others are hidden from AI

When user leaves:

they are automatically deactivated

🔥 Why this is important

This solves:

token bloat (don't send irrelevant tools)

context confusion

security (actions only exist where valid)

💡 Even better: lifecycle hooks

```
ai.onRouteChange((route) => {  
  if (route === "/cart") {  
    ai.register(checkoutAction);  
  }  
});
```

But internally, the SDK should still normalize this into scoped manifests.

⚙️ 2. Parameter Clarification Strategy

Who handles missing required fields?

❌ Do NOT put this logic in system prompt alone

If Gemini handles it:

inconsistent behavior

hard to debug

unpredictable UX

✗ Also don't rely only on SDK blocking execution

That breaks conversational flow.

✓ Best architecture: Hybrid validation + AI clarification loop

Step 1 — SDK defines schema

```
ai.register({  
  
  bookTable: {  
  
    params: {  
  
      guests: { type: "number", required: true },  
  
      time: { type: "string", required: true },  
  
      date: { type: "string", required: true }  
  
    }  
  
  }  
  
});
```

Step 2 — AI tries to call tool

If incomplete:

```
{  
  
  "tool_use": {  
  
    "name": "bookTable",
```

```
"args": {  
  "guests": 2  
}  
}
```

Step 3 — SDK intercepts BEFORE execution

SDK checks:

missing time

missing date

Step 4 — SDK sends structured “clarification request” back to Gemini

```
{  
  "tool_validation": {  
    "name": "bookTable",  
    "missing": ["time", "date"]  
  }  
}
```

Step 5 — Gemini asks user naturally

“What time and date would you like the reservation?”

🔥 Why this is the best approach

You get:

deterministic validation (SDK layer)

natural conversation (AI layer)

no hallucinated execution

💡 Key rule

AI decides what to ask

SDK decides what is valid

🧱 3. Code-Truth Sync (SDK → Dashboard)

How does the dashboard know what exists in code?

✅ Yes — you absolutely need a Manifest system

This is the backbone of your platform.

💬 Step 1 — SDK generates a manifest

On initialization:

```
ai.init({  
  
  appId: "resto-app"
```

```
});
```

SDK sends:

```
{  
  "type": "manifest",  
  "actions": [  
    {  
      "name": "bookTable",  
      "params": ["guests", "time", "date"],  
      "scope": "global"  
    },  
    {  
      "name": "addToCart",  
      "params": ["itemId", "quantity"],  
      "scope": "/menu"  
    }  
  ]  
}
```

 Step 2 — Backend stores versioned manifest

You now have:

action registry snapshot

version history

environment mapping

 Step 3 — Dashboard reads from backend

Dashboard becomes:

a visual reflection of SDK reality

not the source of truth

 Critical design decision

You must choose:

 1 Best (recommended)

Code = source of truth

Dashboard = visualization + override layer

 2 Alternative (riskier)

Dashboard = source of truth

SDK pulls config

 This leads to sync conflicts later

💡 Pro upgrade: diff system

If code changes:

+ addToCart added

- checkout removed

Dashboard shows:

“New actions detected in code”

⚡ Final architecture (clean version)

🔗 1. Dynamic actions

scoped registration

lifecycle-aware

route-based activation

⚙️ 2. Parameters

SDK validates schema

AI handles conversational clarification

loop between SDK ↔ Gemini

🧱 3. Code sync

SDK emits manifest

backend stores versions

dashboard renders from manifest

code remains source of truth

Edge Cases & UX

1. 🙋 1. “I Don’t Know” Fallback

Should the AI behave like a general assistant or stay app-limited?

❌ Avoid: fully generic assistant mode

If your AI suddenly becomes ChatGPT-like:

users lose mental model of the product

it feels unpredictable

it can suggest actions your app doesn’t support

harder to secure and debug

✅ Best approach: bounded general intelligence

Think of it as:

“AI that can only act inside your app, but can still talk naturally when it doesn’t know.”

💡 Behavior model

1. First try to match an action

if match → execute tool

2. If no match:

AI switches to:

“I can only help with what this app supports”

Example UX (food ordering app)

User:

“Book me a flight to Paris”

AI:

“I can only help with ordering food, managing your cart, and reservations. Would you like to see today’s menu instead?”

 Key principle

The AI can talk freely, but it cannot act outside defined capabilities.

 Pro upgrade (highly recommended)

Let developers define a fallback:

```
ai.setFallback({  
  message: "I can only help with ordering food, reservations, and your cart.",  
  suggestions: ["Show menu", "View cart", "Track order"]  
});
```

⚠️ 2. Confirmation UI for Dangerous Actions

Who builds the confirmation modal?

❌ Don't force developers to always build it

That leads to:

inconsistent UX

security mistakes

missing confirmations in some apps

✅ Best approach: SDK-provided default confirmation layer

But with override capability.

🧠 How it should work

Step 1 — action is marked dangerous

```
ai.register({  
  deleteAccount: {  
    dangerous: true  
  }  
});
```

Step 2 — SDK intercepts execution

Instead of immediately calling backend:

👉 SDK shows built-in confirmation UI:

“Are you sure you want to delete your account?”

Buttons:

Cancel

Confirm

Step 3 — only after confirmation:

tool executes

AI continues flow

🔥 Why this is important

This ensures:

consistent safety UX across all apps

zero reliance on developer discipline

fewer catastrophic mistakes

💡 But still allow customization

Developers can override:

```
ai.ui.confirmation({  
  
  deleteAccount: CustomModalComponent  
  
});
```

🧠 Rule of thumb

Safety UI should be built-in, but not locked-in.

🌐 3. Multi-Language Switching

Auto-detect or explicit language setting?

❌ Don't rely purely on auto-detection

Mid-conversation switching leads to:

wrong translations

inconsistent responses

confusing UX in mixed-language environments

✗ Don't force manual switching only

That makes the system feel rigid.

✓ Best solution: hybrid language system

🧠 Layer 1 — default language (developer-defined)

```
ai.setLanguage("en");
```

🧠 Layer 2 — auto-detection (for input only)

SDK can detect:

user input language

voice language

But only as a hint, not a hard switch.

🧠 Layer 3 — session-level locking (important)

Once conversation starts:

AI sticks to chosen language unless user explicitly changes

Example:

“Switch to Spanish”

Then:

full session switches

🔥 Best UX behavior

Case 1: Single-language app

no switching needed

stable responses

Case 2: Mixed-language input

User:

“I want burger please” + Amharic sentence

AI:

responds in default language

optionally mirrors user language if requested

💡 Advanced option (very powerful)

Let developers enable:

```
ai.enableMultilingual({  
  mode: "adaptive",  
  supported: ["en", "es", "fr"]  
});
```

Modes:

strict → fixed language

adaptive → follows user

manual → developer controls

THE TOP LAYER PHASES TO IMPLEMENT ALAN AI

This is a massive vision, and the "Capability-first" (Intent-less) approach is exactly how high-end AI engineering is moving. You are effectively building an Agentic Operating System for the Web.

Here is the Top-View Action Plan to build the Doweit Voice SDK and its ecosystem. I have broken this into 4 logical phases.

Phase 1: The "Nerve Center" (Backend & Protocol)

Before we build the SDK, we need a way for the SDK to talk to Gemini Live securely and manage the "Manifest" (the list of capabilities).

1. **WebSocket Proxy Server:** Create a specialized Next.js/Node.js route that handles the WebSocket connection from the end-user's browser. It will securely inject your API keys and the "System Instructions" before forwarding the stream to Gemini Live.
2. **Manifest Storage:** Update your database schema to store "App Manifests." This will track every function a developer has registered in their code so the Dashboard can visualize them.
3. **Tool-Calling Loop Logic:** Build the backend logic that intercepts Gemini's `tool_use` request, sends it to the SDK, waits for the result, and feeds it back to Gemini.

Phase 2: The "Brain Stem" (Core SDK)

This is the invisible logic that the developer installs via NPM.

1. NPM Package Scaffolding: Set up the doweit-voice package structure.
2. The Registry: Build the `ai.register()` and `ai.setState()` methods.
3. The Manifest Emitter: Logic that gathers all registered functions on app startup and sends a "Snapshot" to the Doweit Backend so the Dashboard stays in sync.
4. Action Executor: The listener that receives "Call function X with Args Y" from the backend and triggers the actual JavaScript function in the developer's app.
5. Validation Layer: The logic that checks if required parameters are present before letting the AI execute a function.

Phase 3: The "Face" (The Widget & UI)

This is the beautiful, customizable bubble that the end-user sees.

1. The Voice/Text UI: A pre-designed, responsive React component (the bubble + chat window) that handles audio visualization, chat history, and text input.
2. State Visualizer: Built-in UI feedback for "AI is thinking," "Action executing," and "Confirmation required."

3. Barge-in Logic: Frontend audio management that kills the "AI speaking" audio context the moment the "User speaking" status is detected.

Phase 4: The "Control Tower" (Doweit Dashboard)

Where the developer manages, tests, and deploys.

1. The Alan AI Section: A new dashboard area for managing these specific embedded agents.
2. Visual Manifest Viewer: A UI that lists all functions detected in the developer's code (pulled from the Manifest Emitter).
3. Live Playground: A side-by-side view where the developer can talk to their app in real-time and see exactly which functions are being triggered and what state is being sent.
4. Version & Publish Control: The Draft → Staging → Production deployment pipeline.

Immediate First Step: The "Capability Protocol"

I suggest we start with Phase 1 & 2 combined: Defining how the SDK registers a function and how the Backend sends that to Gemini.

If we don't have the "Manifest" system right, the Dashboard will just be a static form. If we have the Manifest system, the Dashboard becomes a dynamic

reflection of the developer's code.

Shall we start by defining the actions and app_manifest database schema and the basic ai.register logic for the SDK?